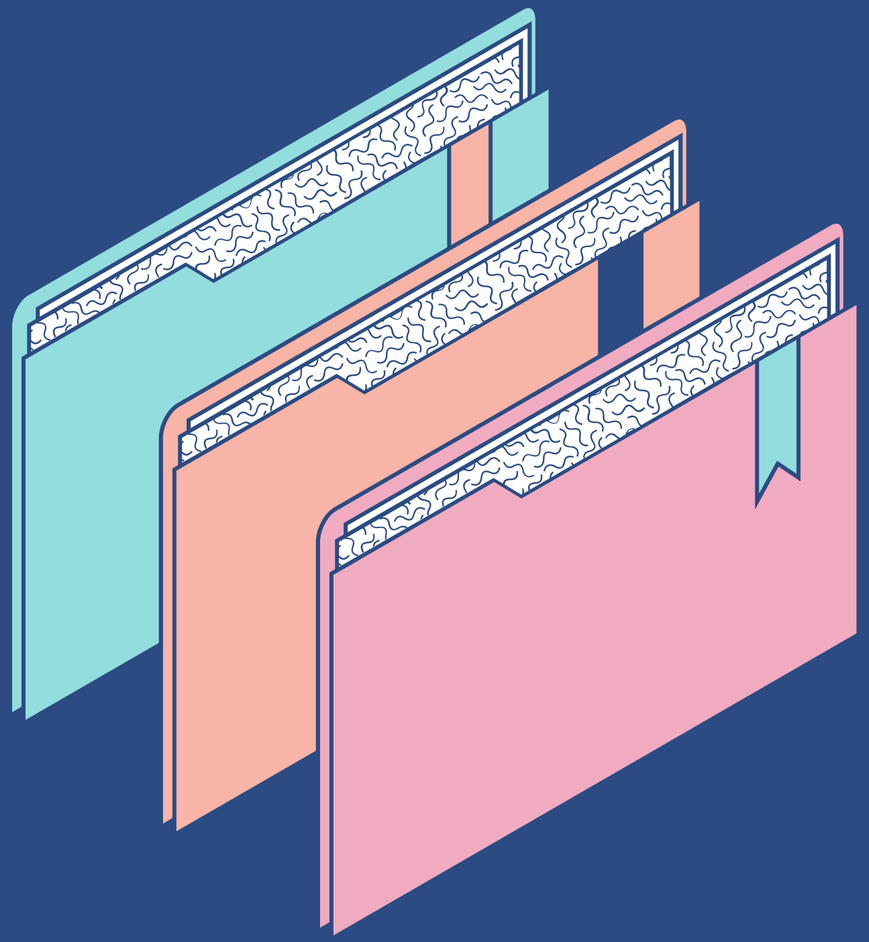


Evaluation AI

Полный курс по оценке качества работы AI решений

Автор курса Мешков Александр



Структура курса

Лекция 1. Основы evaluation и Python
Лекция 2. Оценка ML/DL решений
Лекция 3. Evaluation больших языковых моделей (LLM)
Лекция 4. Evaluation RAG систем
Лекция 5. Evaluation AI агентов и инструментальных систем
Лекция 6. Evaluation мультимодальных AI систем
Лекция 7. Conversational AI и многоходовые диалоги
Лекция 8. Bias, safety и responsible AI evaluation

Чему вы научитесь

ТЕОРЕТИЧЕСКИЕ И ПРАКТИЧЕСКИЕ НАВЫКИ ОЦЕНКИ AI

Понимать принципы работы различных AI приложений

В ходе обучение вы будете работать с системами как класса ML/DL, так и AI приложениями на базе генеративного ИИ (OpenAI, DeepSeek, и другие), включающие ИИ чат-боты, генерация картинок, видео и другие.

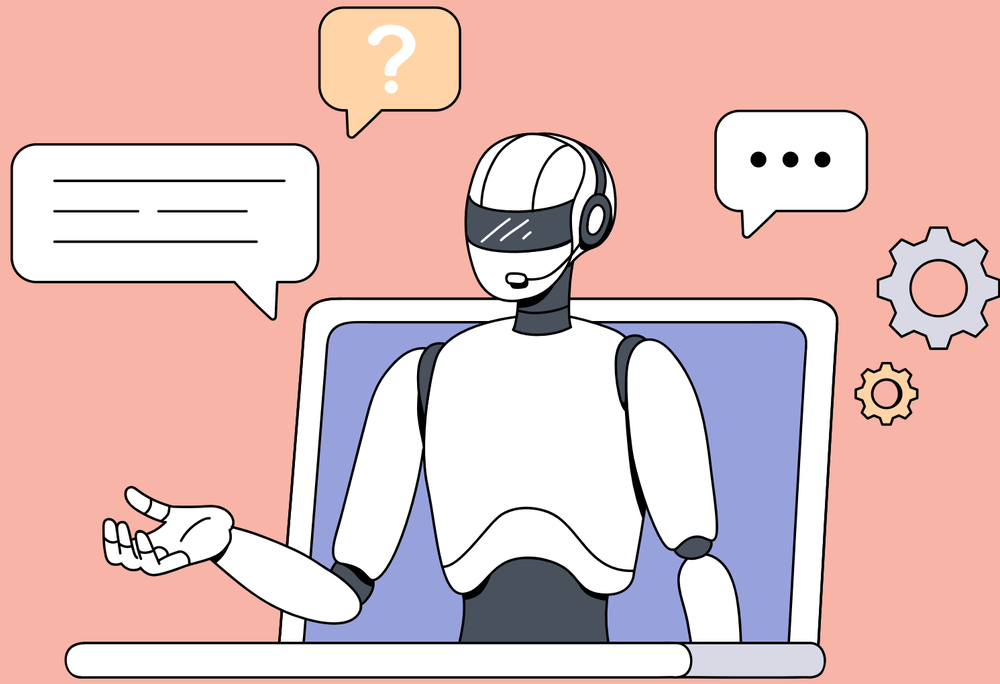
Определять области измерения (метрики) для каждого класса AI систем и создавать датасеты для оценки

В ходе курса мы будем разбирать большое количество метрик, начиная с F1 Score, ROC-AUC для систем ML и заканчивая сложными метриками оценки AI Agentic приложений, а также будем изучать подходы к правильному формированию датасетов для оценки AI

Практически выполнять оценку AI решений

В ходе курса мы будем работать с популярными фремворками, такими к scikit-learn, TensorFlow/Keras, Deepeval, Ragas, HuggingFace, LangChain Evaluation и многими лругими.

Формат обучения



Участие в онлайн лекциях один раз в 2 недели

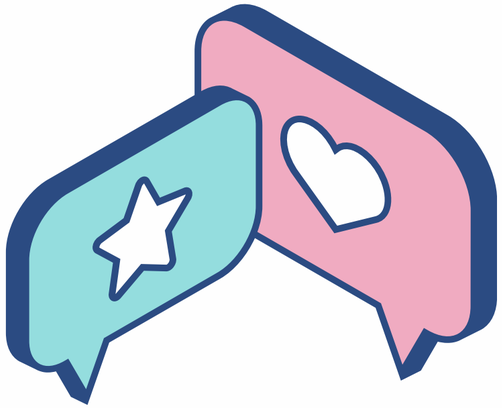
Все лекции проходят в живом формате, где у вас есть возможность задавать мне любые вопросы

Выполнение практических заданий на реальных AI системах

Для получения практических навыков мы будем использовать реальные датасеты и AI системы, а также популярные фреймворки

Общение в чате по любым вопросам, связанными с курсом или практикой

Не стесняйтесь спрашивать и задавать вопросы! Это позволит вам лучше усвоить материал и закрепить полученные знания.





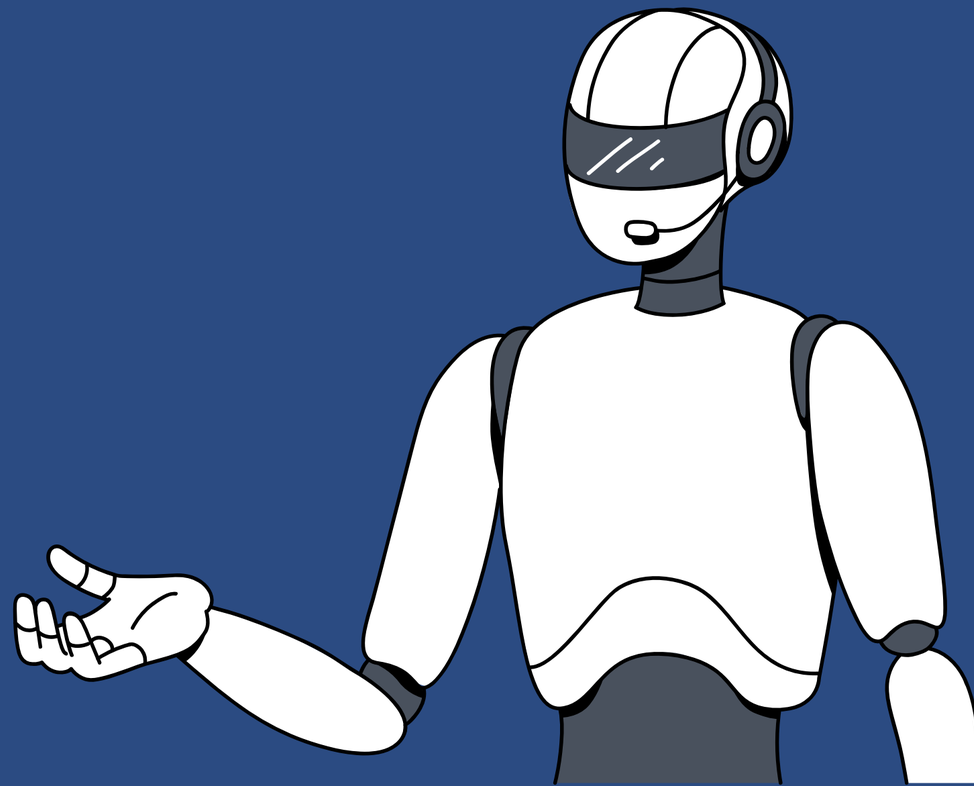
Что потребуется для обучение?

- Время и желание учиться!
- Установленный редактор кода VS Code и Jupyter NoteBook
- Установленный python для проведения evaluation
- Установленный docker desktop и GIT для запуска приложений
- Хорошее настроение!

Лекция 1: Основы AI evalaution и python

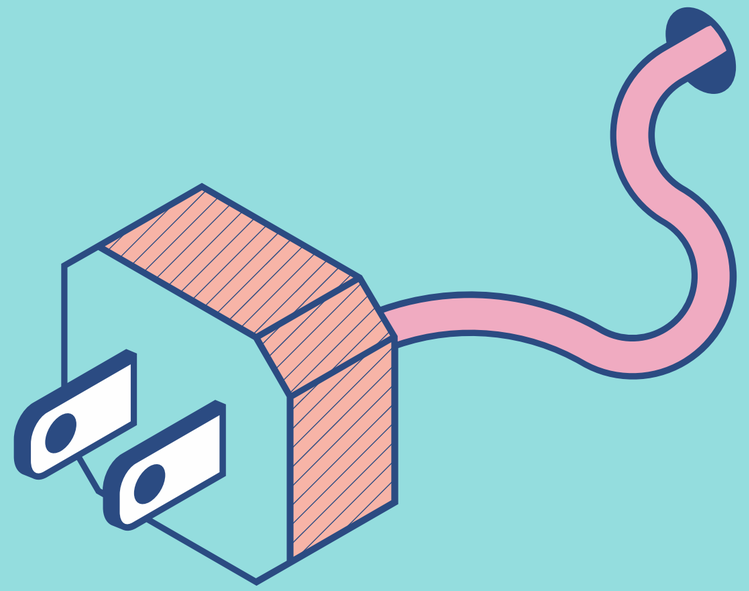
ПЛАН ЛЕКЦИИ:

1. Что такое AI
2. Классификация AI
3. Что такое evaluation и зачем он нужен
4. Сложности в оценке AI решений
5. Критерии правильной оценки
6. Основы Python



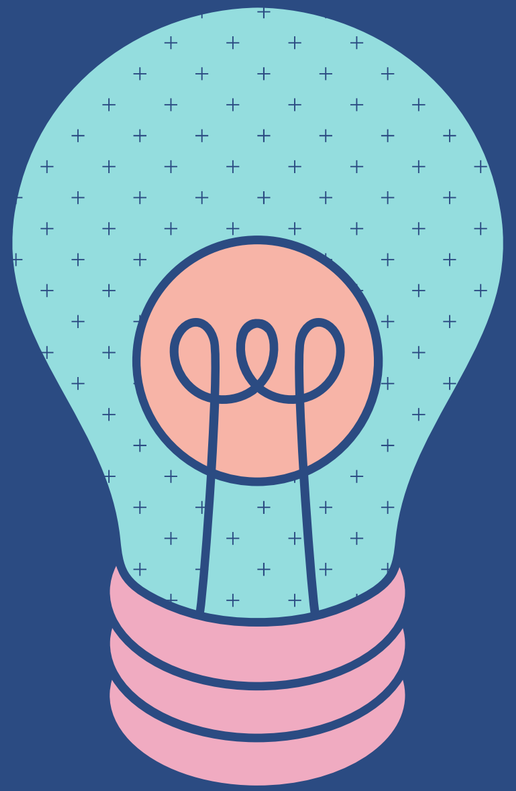
Что изменилось за последние годы?

Современный AI переживает беспрецедентную революцию. Мы перешли от простых алгоритмов классификации к системам, которые могут писать код, создавать изображения, вести сложные диалоги и даже управлять роботами. Эта сложность принесла с собой новые вызовы в области оценки качества.



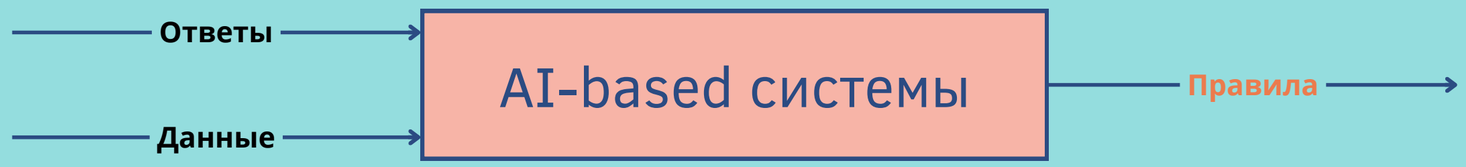
Искусственный интеллект (ИИ) – это область компьютерных наук, занимающаяся разработкой систем, которые могут имитировать человеческий интеллект, включая способность к обучению, решению проблем и принятию решений.

Проще говоря, ИИ – это способность машины или компьютерной системы выполнять задачи, для которых обычно требуется человеческий интеллект.



Отличие от классических систем





Важно! На вход в **классической разработке** у нас есть правила и данные, которые обрабатываются системой и на выходе мы получаем ответ от системы.

На вход **в системе на базе AI** подаются ответы и данные, и получаются правила, по которым система должна обрабатывать новые данные в будущем.

Типы AI

Важно! Искусственный интеллект – это достаточно широкая отрасль, которая в свою очередь охватывает и машинное и глубокое обучение.

Методы машинного обучения позволяют создавать новые типы AI, такие как NLP, Expert system и другие.



Глобальные типы AI

Narrow AI

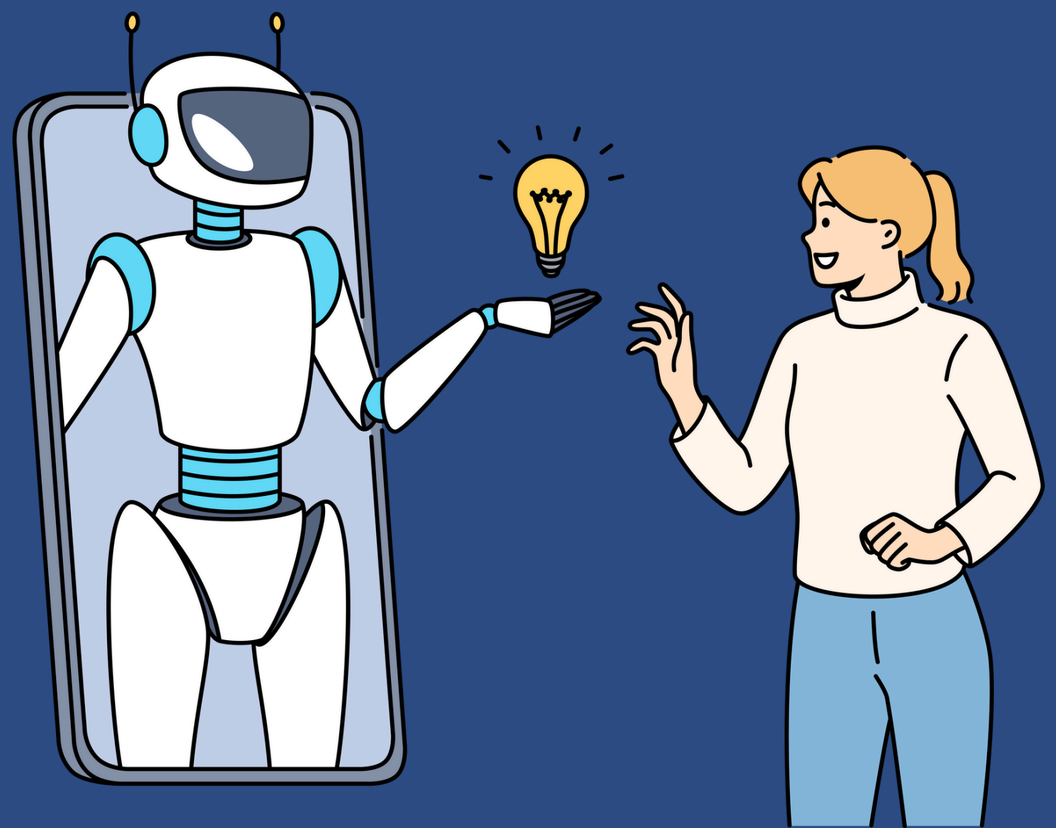
General AI (AGI)

Super AI (strong AI)

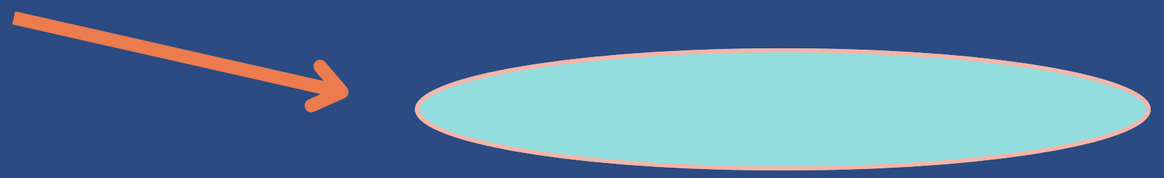


GPT-3.5

Generative AI



Narrow AI



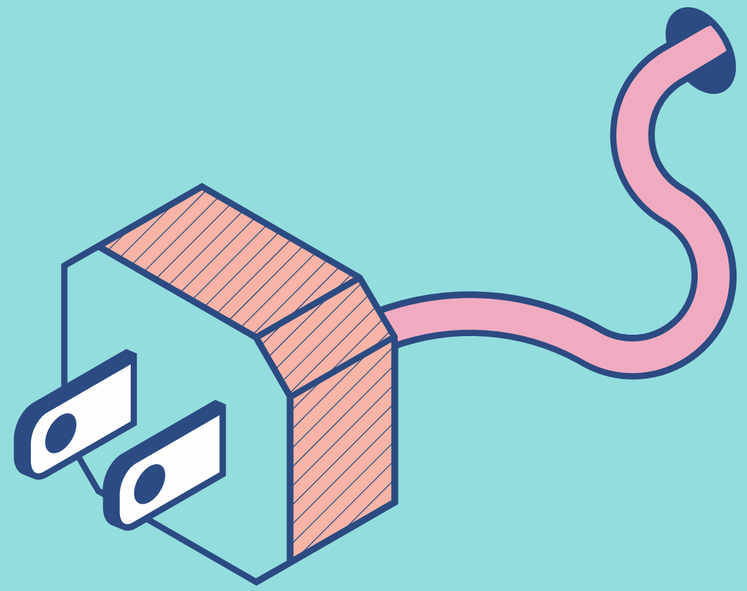
General AI (AGI)

Super AI (strong AI)

Evaluation (оценка) — систематический процесс измерения качества и эффективности AI-модели

Зачем нужен evaluation?

- Сравнение AI решений и выбор лучшего подхода
- Настройка и улучшение AI систем
- Проверка готовности к реальному использованию
- Контроль качества работы AI в реальном мире
- Обеспечение доверия и проверяемости AI решений



Сложности в оценке AI систем

Неправильная оценка AI-систем — это не просто техническая ошибка, а фундаментальная проблема, которая может привести к катастрофическим последствиям в реальном мире.

1 Неправильный выбор задач для оценки

2. Проблемы с метриками и их интерпретацией

3. Проблемы с человеческой оценкой

4. Проблемы воспроизводимости и стандартизации

5. Проблемы с данными для оценки

Критерии правильной оценки AI

1

Пользователь-центричный подход

Правильная оценка всегда начинается с вопроса: кто будет использовать эту систему и для решения каких реальных проблем?

2

Правильный выбор критериев оценки

Выбор критериев оценки — это искусство балансирования между техническими возможностями измерения и практической значимостью для пользователей.

3

Правильная интерпретация результатов

Реальное качество AI-системы не может быть выражено одной цифрой. Правильная оценка измеряет множественные аспекты. Важно понимать, как интерпретировать результаты и принимать решения на основе противоречивых данных.

4

Валидация на репрезентативных данных

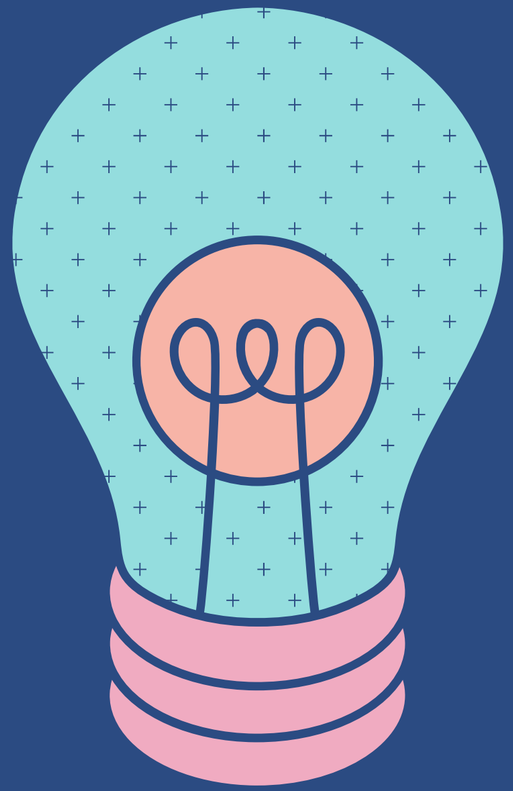
Правильная оценка требует создания тестовых наборов, которые максимально точно отражают реальное использование.

5

Организация человеческой оценки

Правильная организация человеческой оценки начинается с четких инструкций для оценщиков. Критерии оценки должны быть конкретными.

Что нам нужно для оценки AI?



Jupyter Nootbook + python



Установка Python

Windows:

- Перейдите на официальный сайт Python: https://python.org
- Нажмите "Download Python" (скачается последняя версия)
- Запустите установочный файл
- Важно: Поставьте галочку "Add Python to PATH"
- Выберите "Install Now"
- Проверьте установку: откройте командную строку (cmd) и введите

```
python --version
```

Установка Python

macOS:

- Установите Homebrew (если нет): откройте Terminal и выполните:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

- Установите Python через Homebrew:

```
brew install python
```

- Проверьте: python3 --version

Установка Jupyter Notebook

После установки Python выполните в терминале/командной строке:

```
pip install jupyter
```

или

```
pip3 install jupyter
```

Запуск Jupyter Notebook

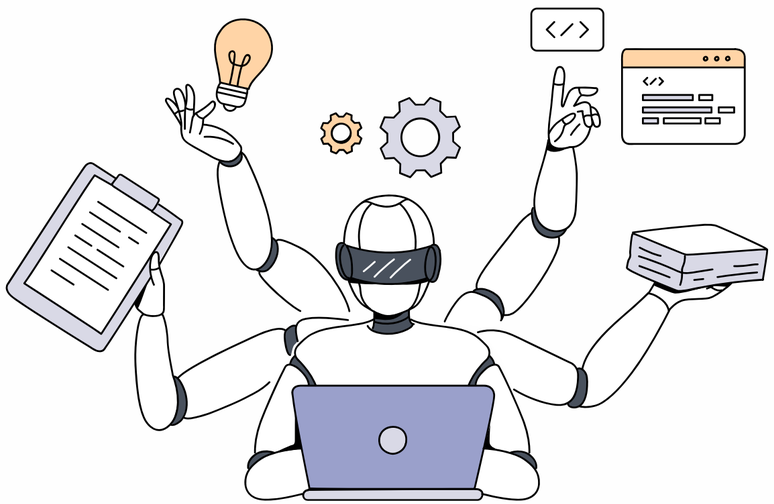
- Откройте терминал/командную строку
- Перейдите в нужную папку: cd путь/к/вашей/папке
- Запустите Jupyter:

```
jupyter notebook
```

- Браузер автоматически откроет интерфейс по адресу http://localhost:8888

Практика

Основы Python



Наши цели

- Изучить основы синтаксиса ЯП Python
- Разобраться, какие есть типы данных, базовые классы, функции и собственные классы

Задание 1.

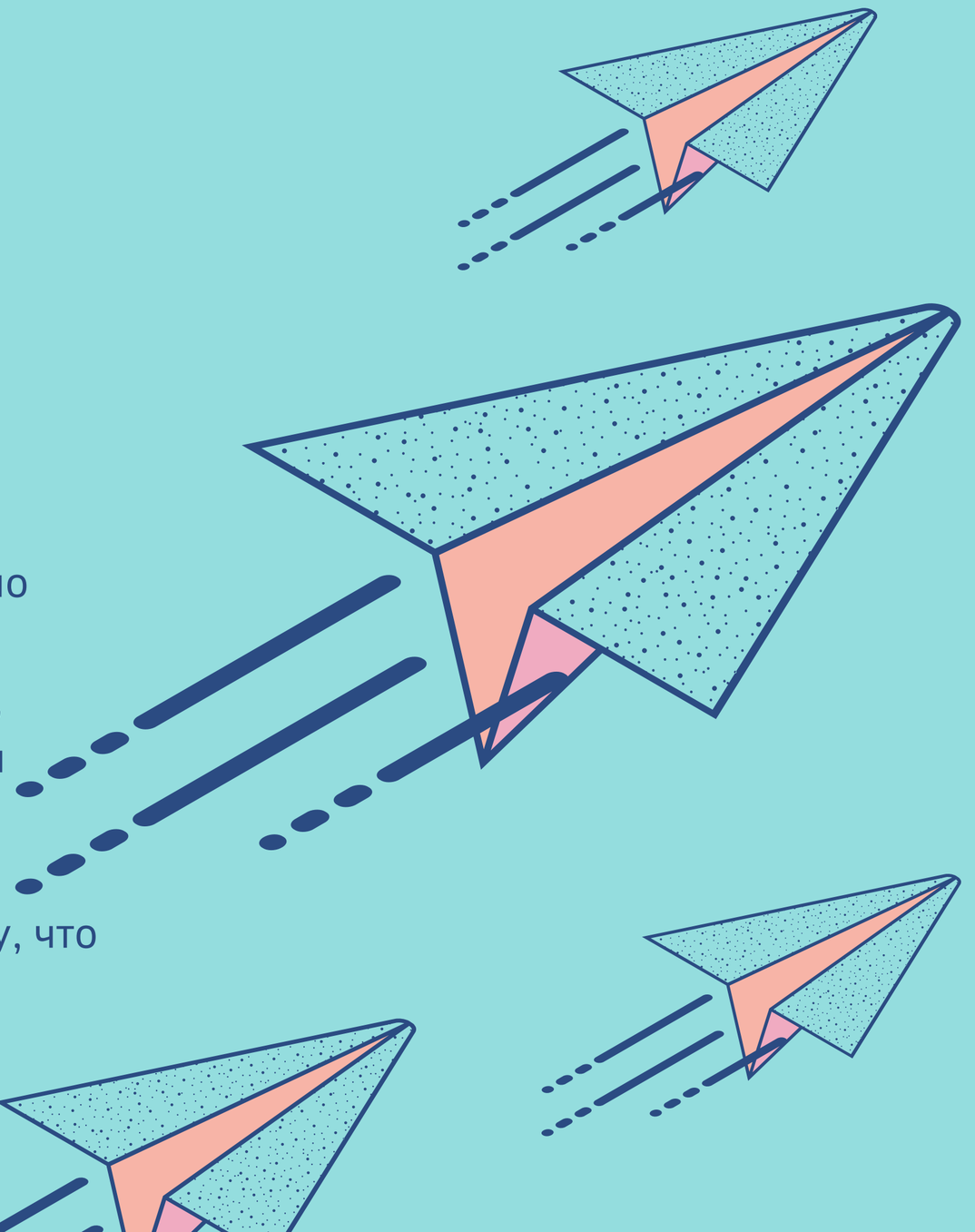
```
# Данные для анализа

predictions = [1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0]

true_labels = [1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1,0, 0, 1]

email_subjects = [ "Поздравляем! Вы выиграли миллион!", "Встреча завтра в 15:00", "Срочно!
Переведите деньги", "Отчет по проекту", "Счет за интернет", "Новости компании", "Кликните здесь
и получите приз!","Приглашение на конференцию", "Специальное предложение только сегодня!",
"График отпусков", "Скидка 90% на все товары!", "Акция! Звоните прямо сейчас!", "Протокол
совещания", "Уважаемый клиент, срочное сообщение", "Семинар по машинному обучению" ]
```

Делаем самостоятельно



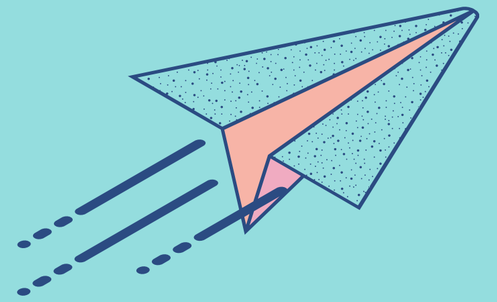
Что сделать:

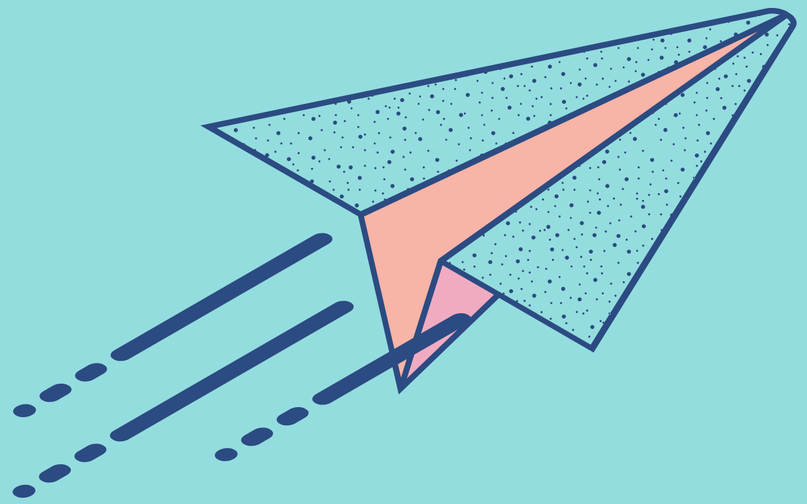
- Подсчитайте общее количество писем
- Найдите количество писем, которые модель классифицировала как спам
- Найдите количество писем, которые действительно являются спамом
- Создайте функцию calculate_accuracy(predictions, true_labels), которая возвращает точность модели
- Вычислите и выведите точность в процентах
- Найдите все случаи, где модель ошиблась
- Для каждой ошибки выведите: номер письма, тему, что предсказала модель, что было на самом деле

Задание 2.

```
experiment_id,model_name,accuracy,training_time,dataset_size
exp_001,BERT-small,0.847,45.2,1000
exp_002,GPT-2,0.923,120.8,1000
exp_003,BERT-large,0.891,180.5,1000
exp_004,Random-Forest,0.756,12.1,1000
exp_005,BERT-small,0.852,44.8,2000
exp_006,GPT-2,0.945,125.3,2000 exp_007,BERT-large,0.912,182.1,2000
exp_008,Random-Forest,0.778,23.4,2000
```

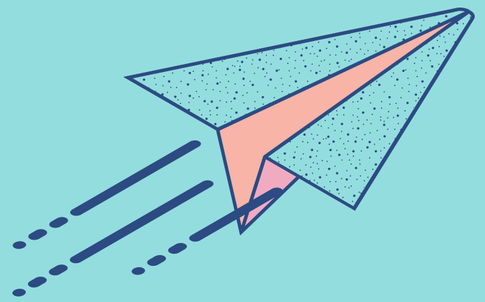
Делаем самостоятельно





Что сделать:

- Создайте файл CSV с данными
- Напишите функцию read_experiments(filename), которая читает файл и возвращает список словарей, каждый словарь должен содержать данные об одном эксперименте
- Создайте список экспериментов с точностью выше 0.9
- Найдите все эксперименты с BERT-моделями (содержат "BERT" в названии)true_labels), которая возвращает точность модели
- Создайте функцию, которая сохраняет сводный отчет в файл report.txt





На сегодня это все!

Вопросы?

